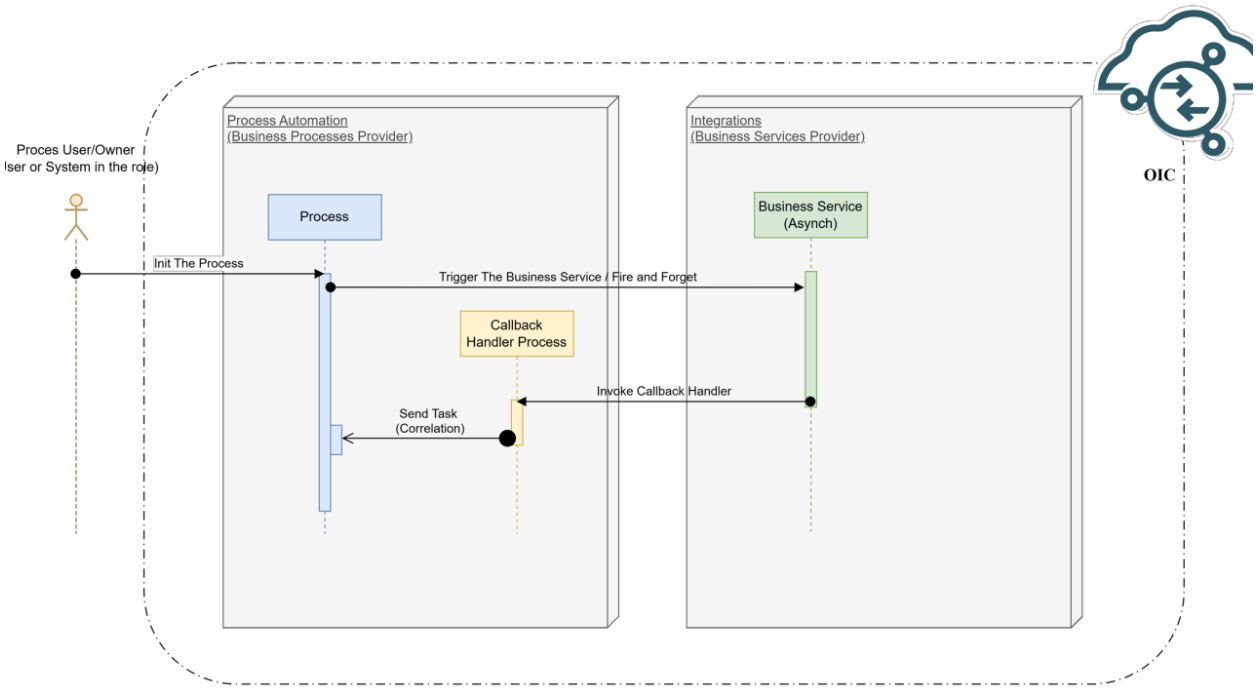


Synchronous → Asynchronous Business Orchestration

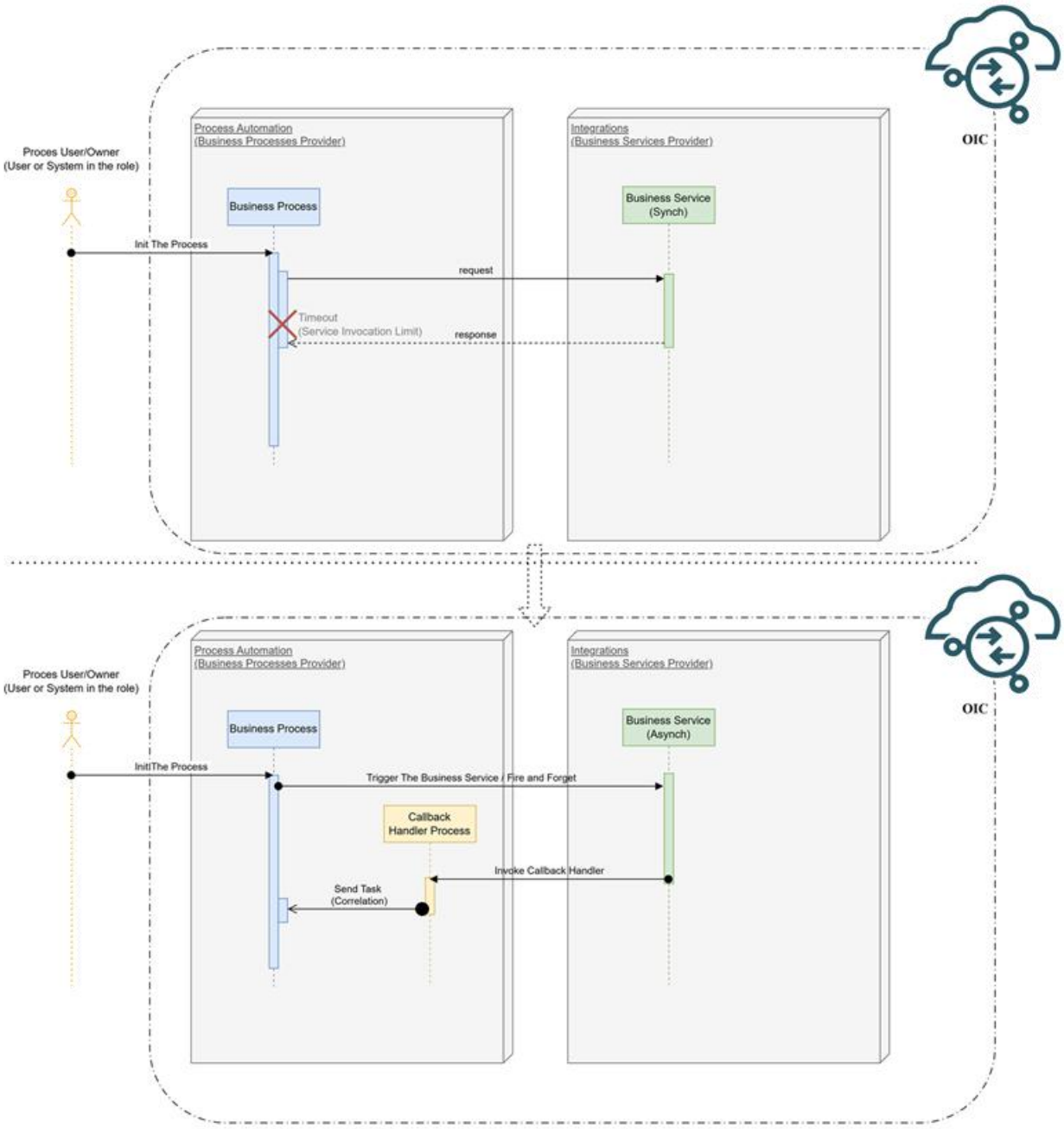
Peter Obert

AI & App Modernization – App Integration and Process Automation Solutions Black Belt @ Oracle | API Management, Cloud Solutions Health-Check, Cloud Solutiond Design and Cloud Solutions Delivery Assistance

March 20, 2026

As business services mature over the time, their response times tend to grow. Synchronous orchestrations that once completed within acceptable limits may begin to time out, causing process failures.

This article walks you through the recipe for re-engineering those orchestrations into a robust asynchronous pattern using Oracle Integration Cloud (OIC) Process Automation and Integrations.



⚠ Problem

Long-running business services exceed synchronous timeout limits, causing faulted process instances in OIC Process Workspace.

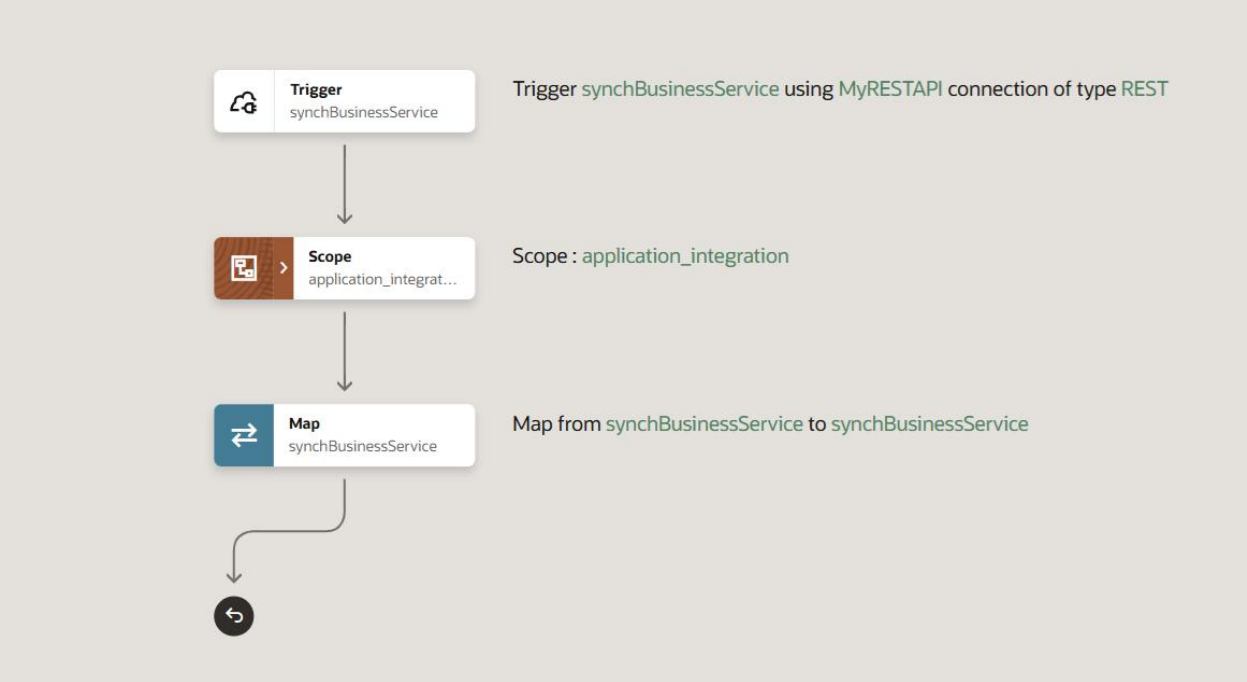
✓ Solution
Refactor synchronous Integration flows and Process Automation orchestrations into an asynchronous callback pattern — achievable in minutes with OIC.

⏱ Limit
The OIC OPA HTTP Connector Timeout is currently 90 seconds. For services that respond in minutes, always design asynchronously.(**OPA Connector Timeout Limit)**

Part 1 — Refactor the Integration (Make It Asynchronous)

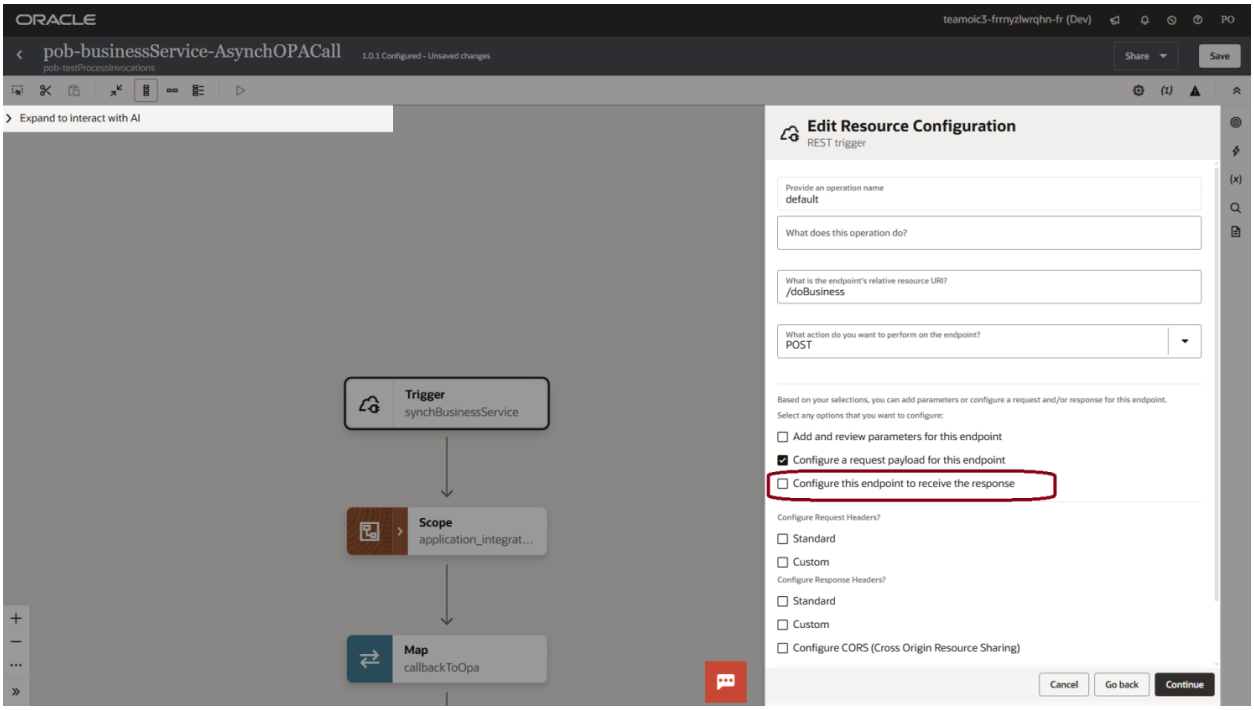
Business services should be wrapped in Integration orchestrations to isolate technology-specific interactions and error handling from the business process itself. This design makes it safe to refactor the integration independently.

1 - Clone the existing synchronous Integration
Create a new version or clone of the integration that currently calls your long-running business service.



2 - Remove the response endpoint

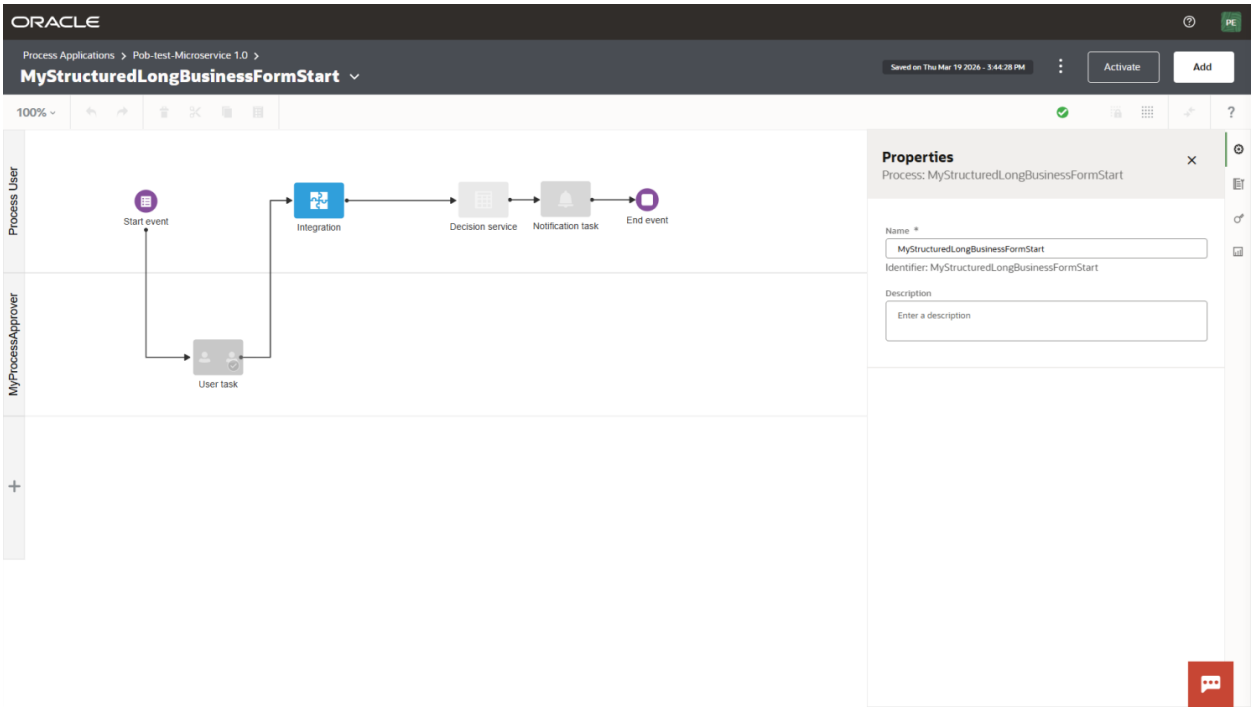
In the cloned integration, open the trigger endpoint configuration and uncheck "Configure this endpoint to receive the response". This removes the Reply action and its associated mapping.



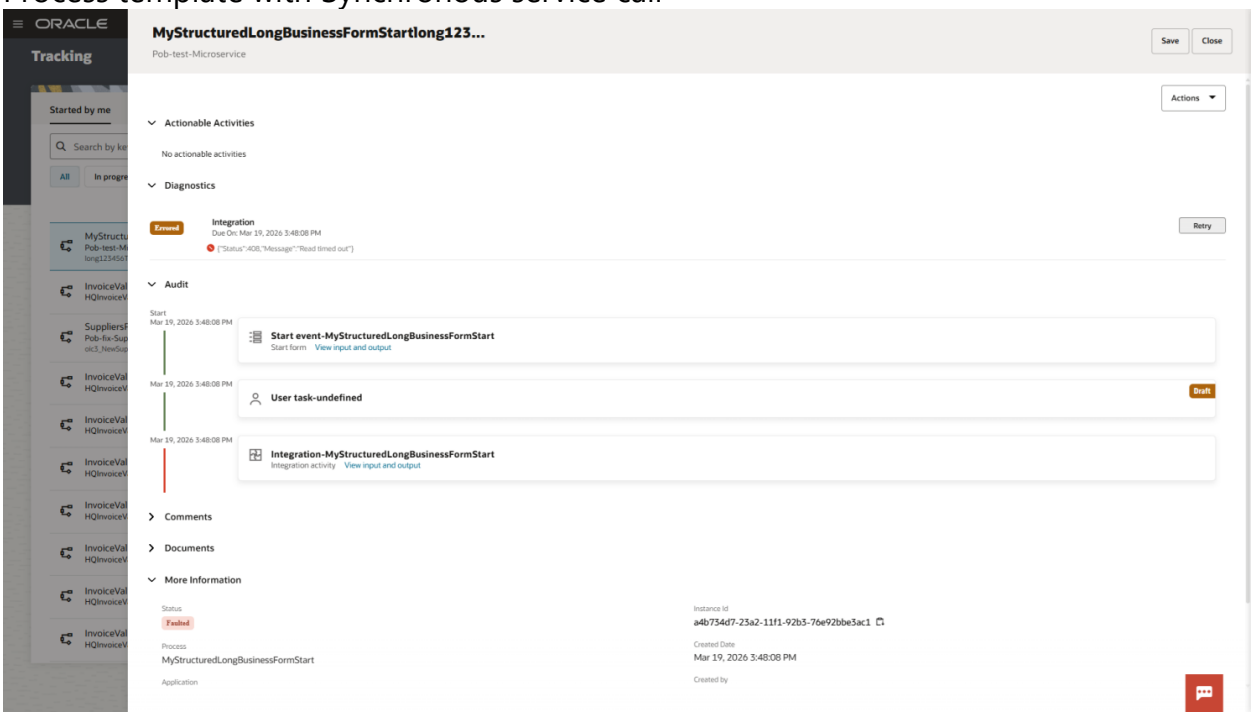
3 - Activate the new integration
Save, validate, and activate the cloned asynchronous integration. Keep the original synchronous version active until the new process is fully validated.

Part 2 — Refactor the Existing Process (Call Integration Asynchronously)

The original process uses an Invoke Integration Service action that calls the synchronous integration. Faulted instances visible in Process Workspace are a symptom of timeout breaches.



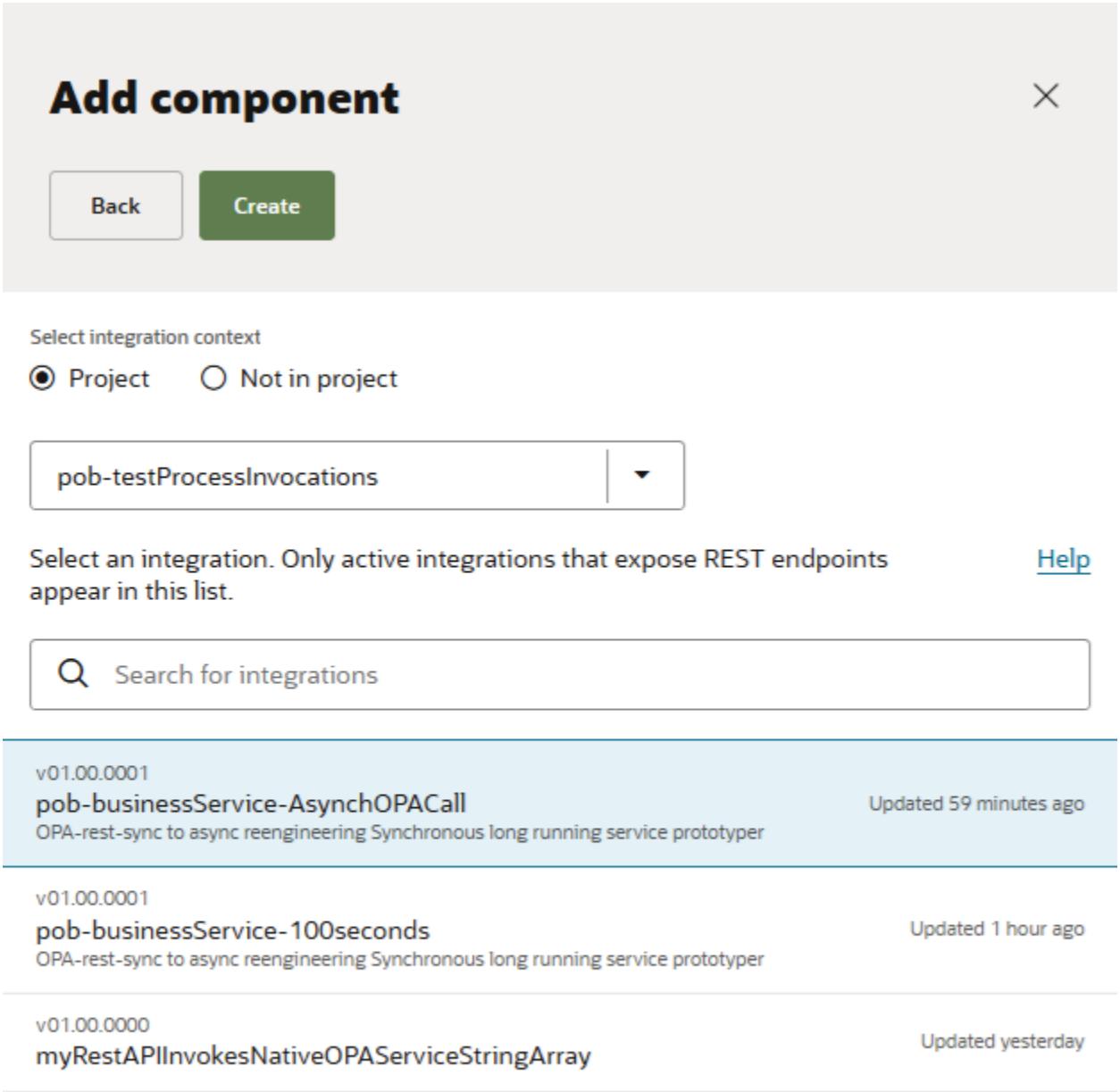
Process template with Synchronous service call



Example of faulted synchronous call business service activity
The refactored process replaces the synchronous call with an async invocation followed by a correlated Receive Task.

1 - Create a connector for the async Integration

In OIC Process Automation, create a new connector pointing to the newly activated asynchronous integration.

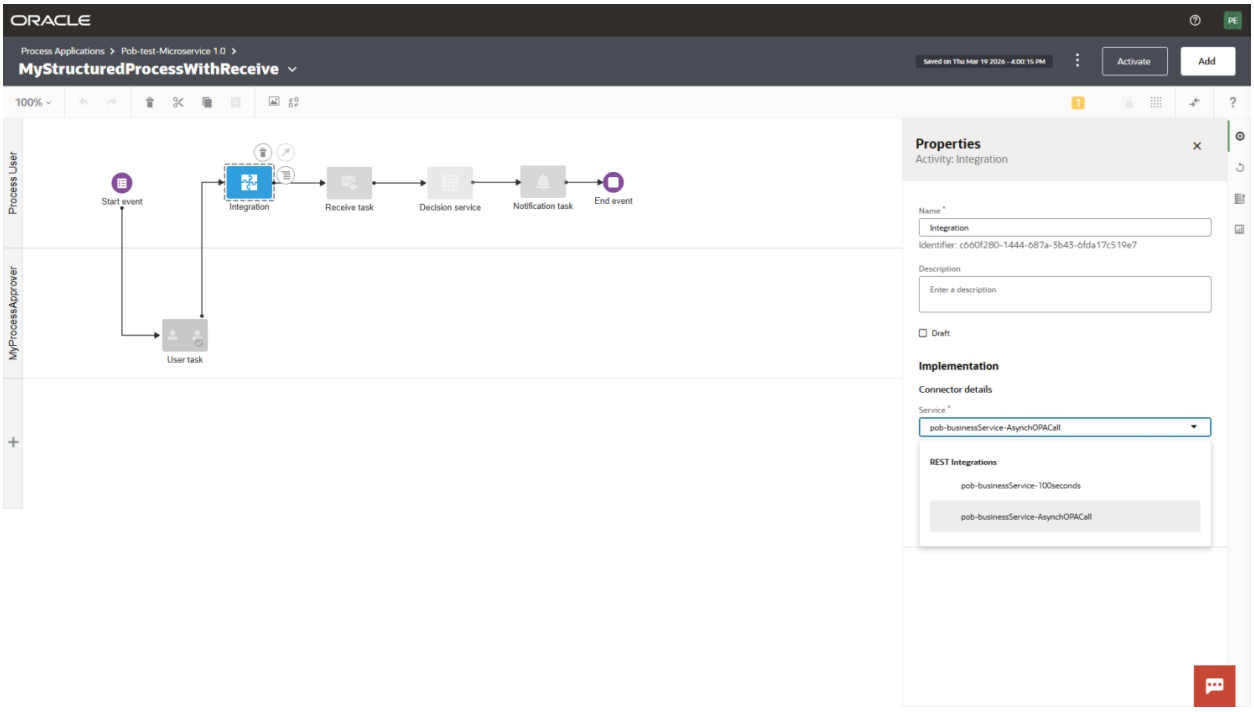


2 - Clone the original Process

Create a clone of the business process. This preserves the original while you build and test the async variant.

3 - Replace the Invoke action with the async connector

In the cloned process, replace the original synchronous Invoke Integration Service action with the new async connector. Note: the invoke will no longer return any output.

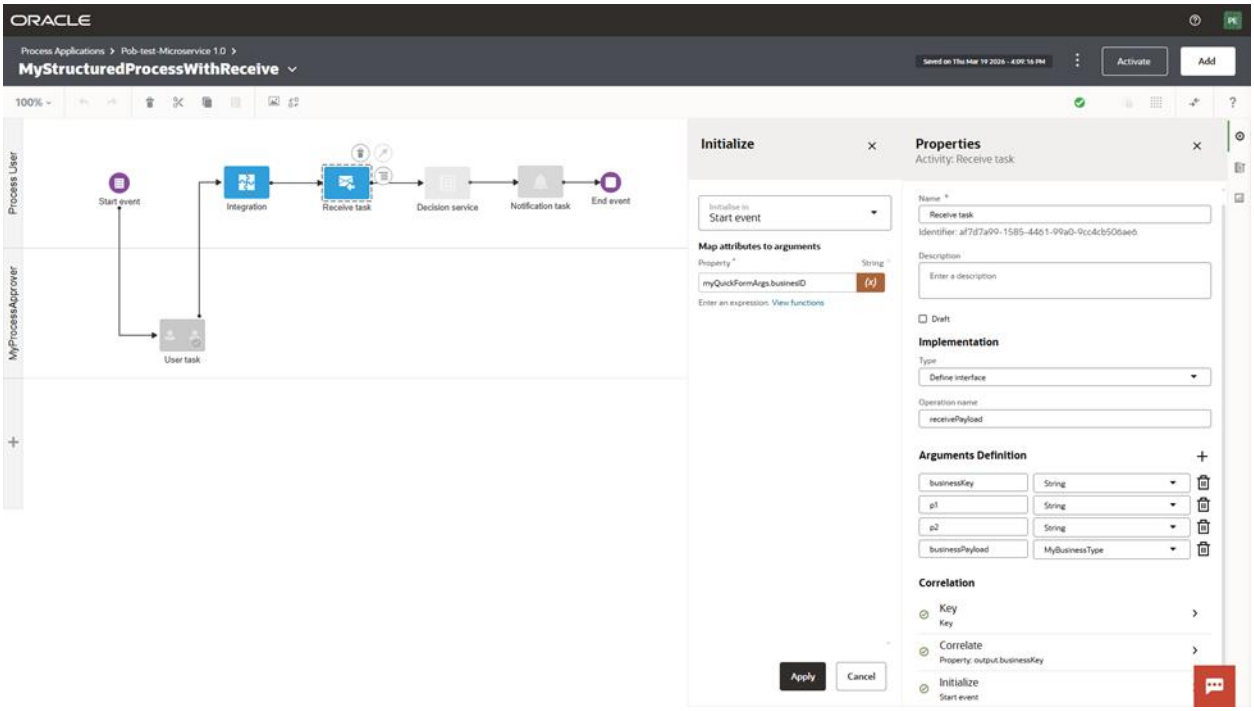


4 - Add a Receive Task after the invoke

Insert a Receive Task activity immediately after the async integration invoke. Configure it to receive the same business object that was previously returned as the integration output.

5 - Set correlation properties

On the Receive Task, define correlation properties so that incoming callback messages are matched to the correct running process instance. This is essential for routing the response to the right execution context.



Part 3 — Implement the Callback Handler (Process Automation)

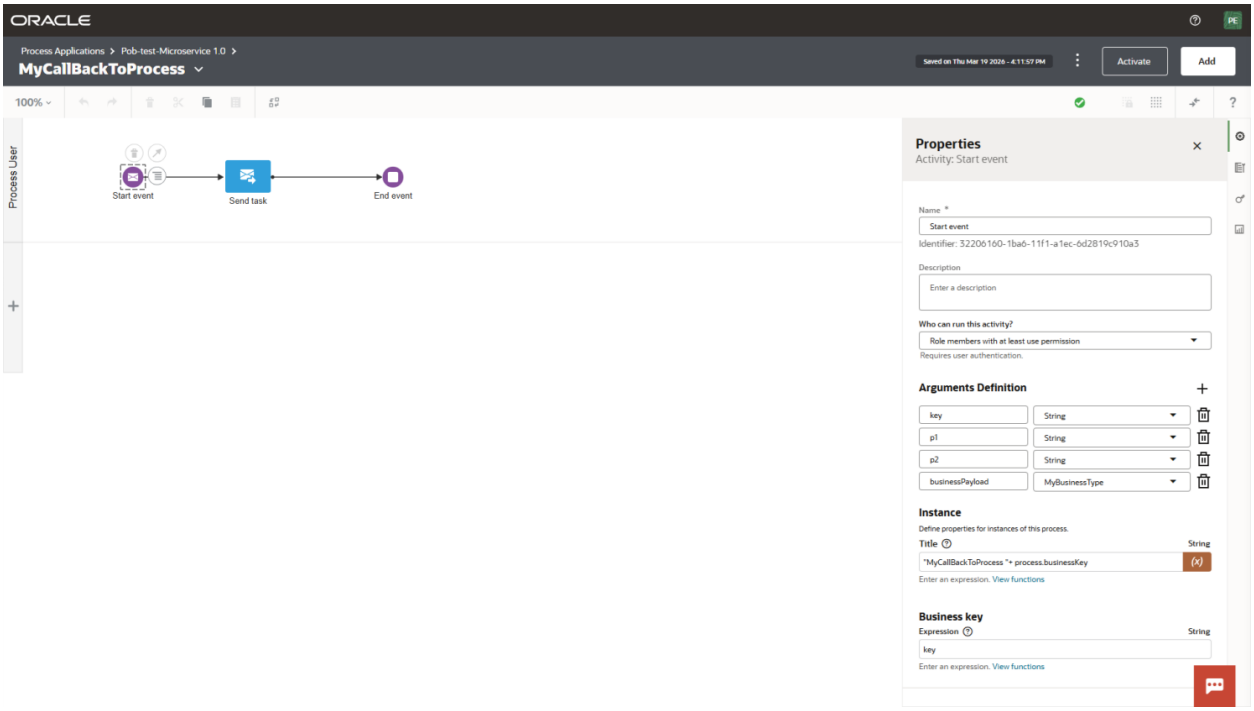
The Callback Handler is a lightweight "microprocess" — a message-initiated Structured Process whose sole responsibility is to deliver the business object to the waiting Receive Task in the main process.

1 - Create a new Structured Process

Create a new message-initiated Structured Process. This will serve as the Callback Handler. Keep it minimal — it should contain only the input business object definition.

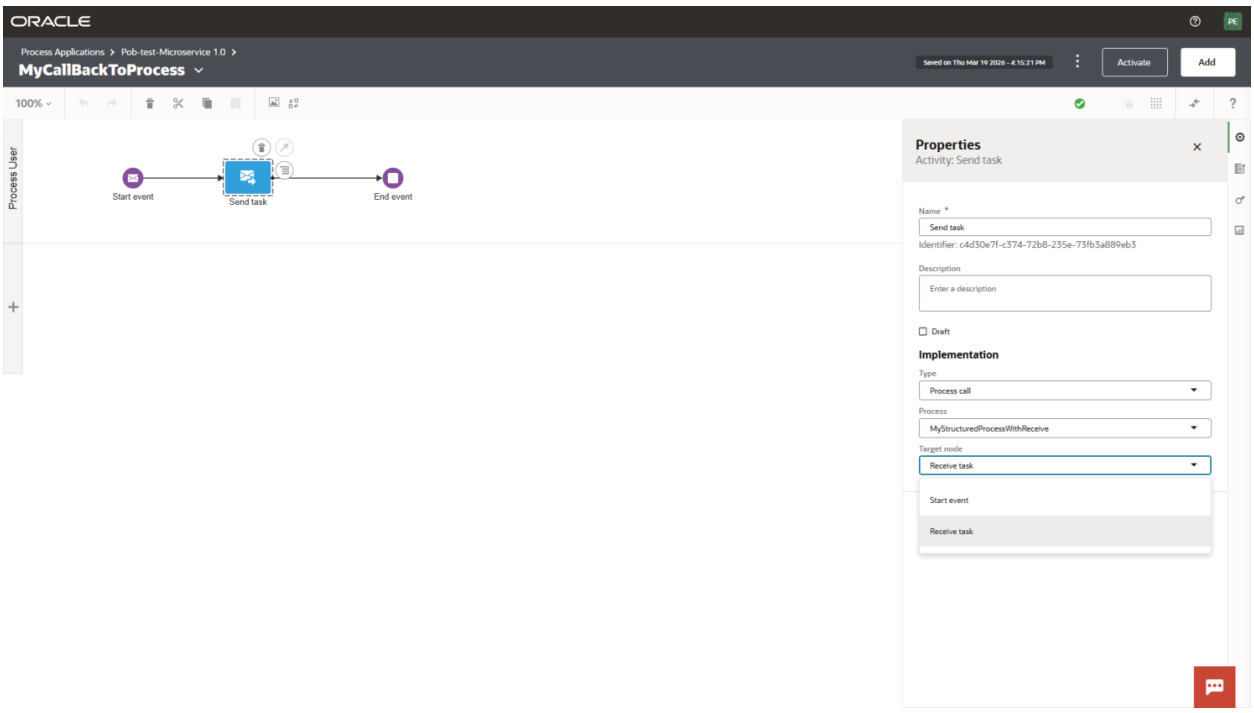
2 - Define the input Business Object

Add the business object as the input to the Callback Handler. This must match the object structure expected by the Receive Task in the refactored business process.



3 - Send the Task to the waiting Process

Configure the Callback Handler to send the task to the Receive Task in the cloned business process. No reply is sent back to the business service — the handler's job ends here.



4 - Activate the Process Application with the Callback Handler
Save, validate, and activate the Callback Handler process before testing the end-to-end flow.

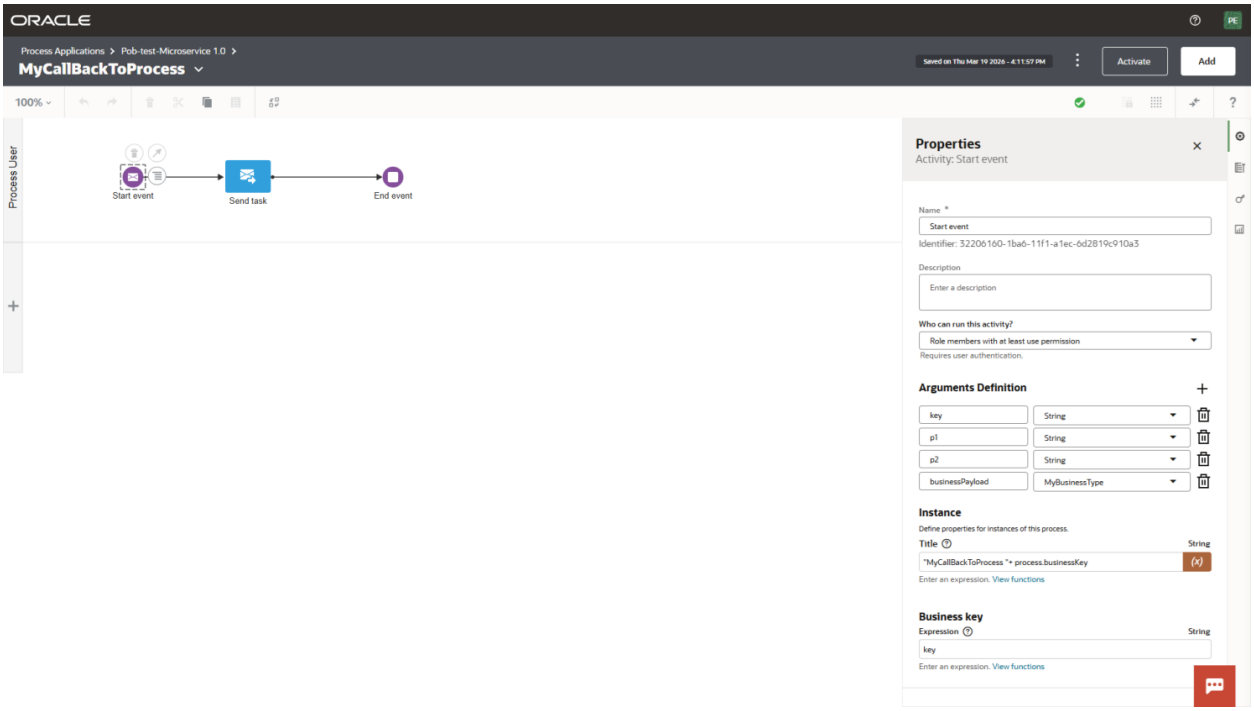
Part 4 - Business Service Callback to “Process Callback Handler Flow” (Integration)

Part 3 — Implement the Callback Handler (Process Automation)

The Callback Handler is a lightweight "microprocess" — a message-initiated Structured Process whose sole responsibility is to deliver the business object to the waiting Receive Task in the main process.

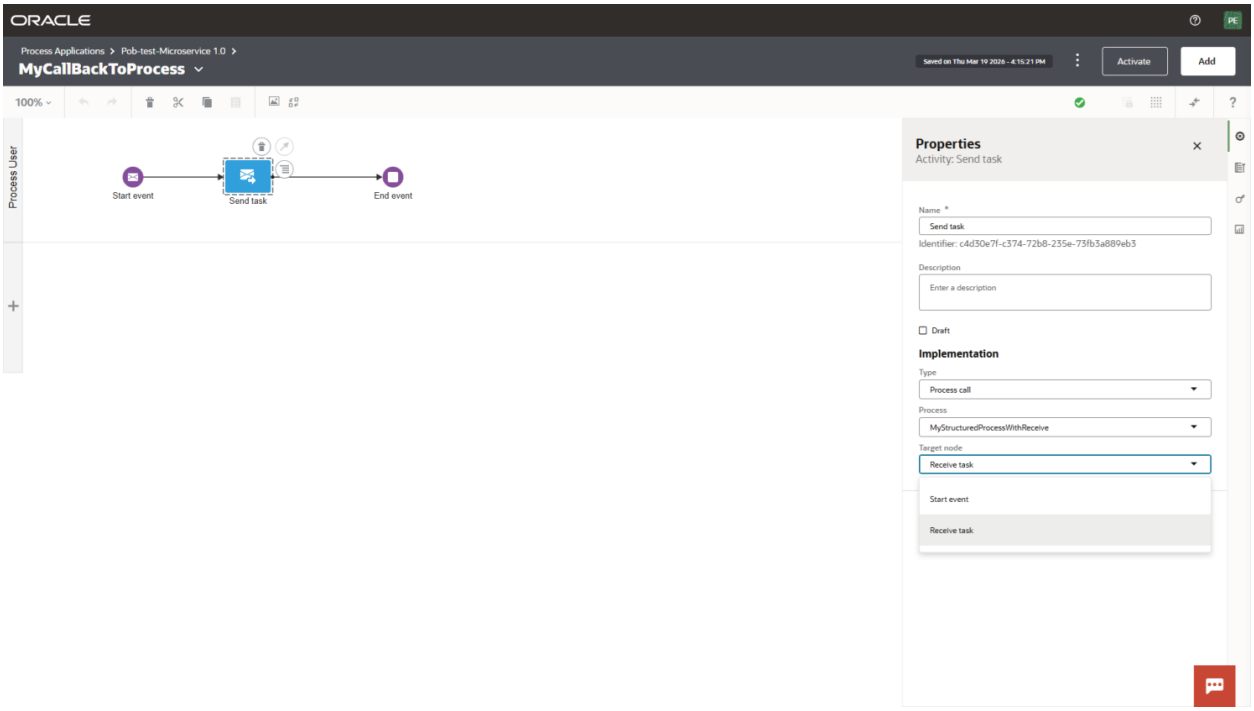
1 - Create a new Structured Process
Create a new message-initiated Structured Process. This will serve as the Callback Handler. Keep it minimal — it should contain only the input business object definition.

2 - Define the input Business Object
Add the business object as the input to the Callback Handler. This must match the object structure expected by the Receive Task in the refactored business process.



3 - Send the Task to the waiting Process

Configure the Callback Handler to send the task to the Receive Task in the cloned business process. No reply is sent back to the business service — the handler's job ends here.



4 - Activate the Process Application with the Callback Handler

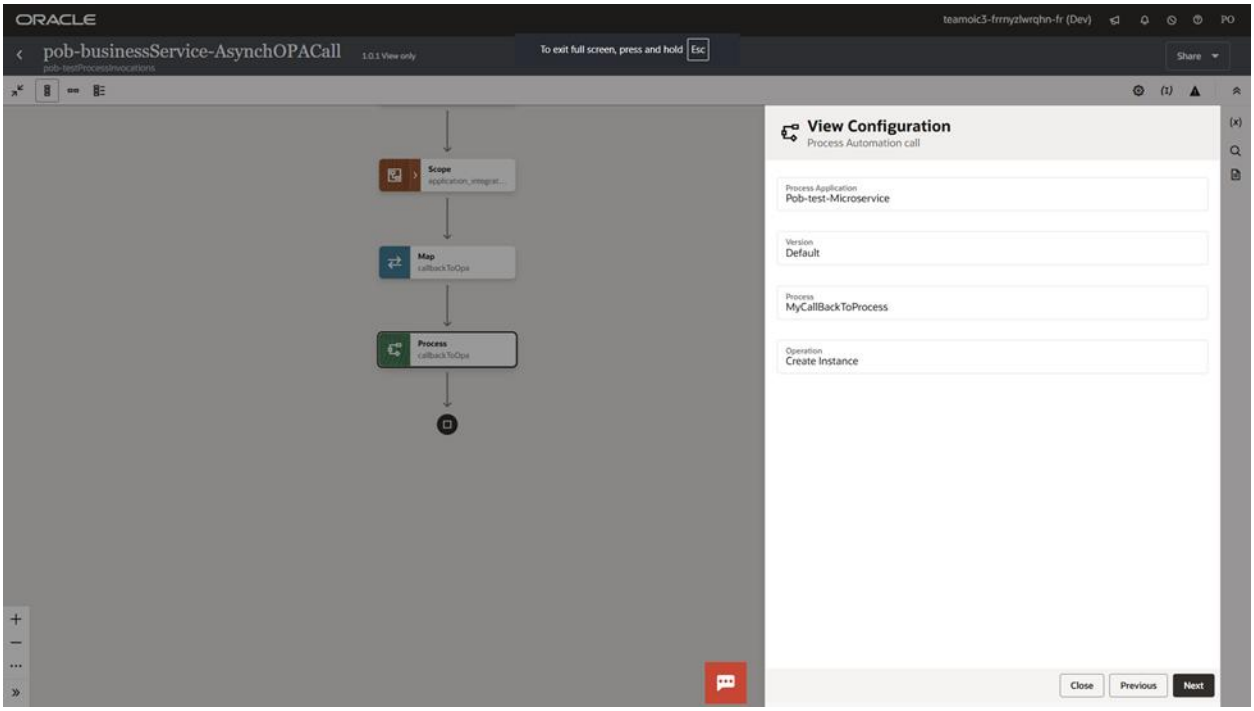
Save, validate, and activate the Callback Handler process before testing the end-to-end flow.

Part 4 - Business Service Callback to “Process Callback Handler Flow” (Integration)

Now we can finish the implementation of the newly created asynchronous Integration flow with invocation of the Process Callback Handler Flow.

1 - Add a Callback invocation

At the end of the async integration flow, add an invocation to the Process Callback Handler Flow (created in Part 3). Map the response payload exactly as the original mapping did — the data object structure remains the same.



Mapping implementation generated by the process invocation will be very the same as it was in the original mapping to populate “response” variable.

2 - Activate the new integration

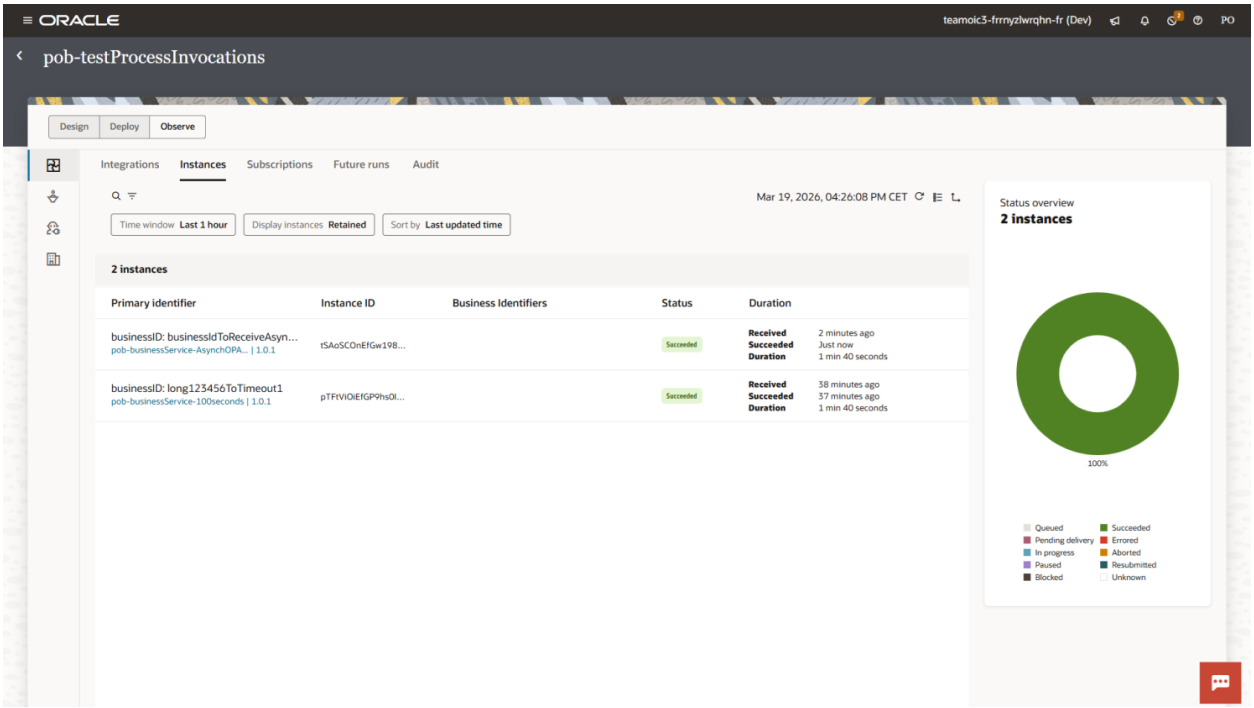
Save, validate, and activate the cloned asynchronous integration. Keep the original synchronous version active until the new process is fully validated.

Part - 5 Testing and Validation

After activating all three components, run an end-to-end test to confirm the refactored flow works correctly.

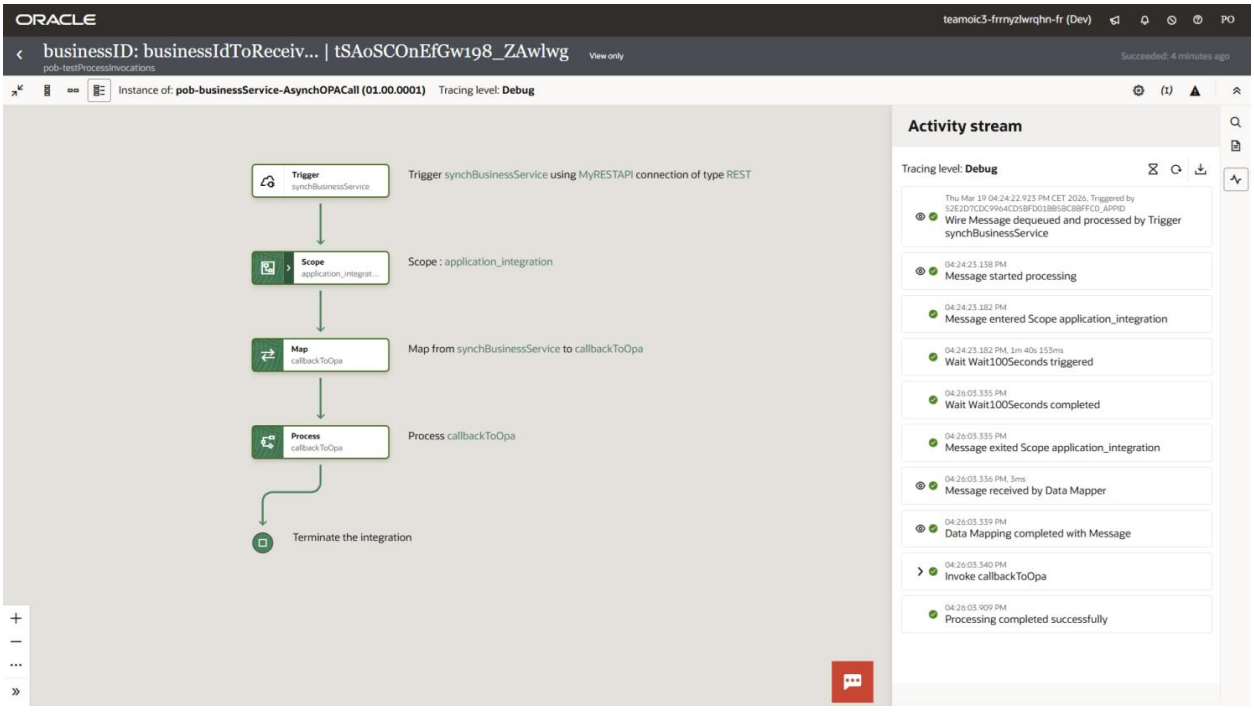
• Initiate the refactored process from Process Workspace or your trigger mechanism.

• Confirm a new asynchronous Integration flow instance is initiated and completes successfully

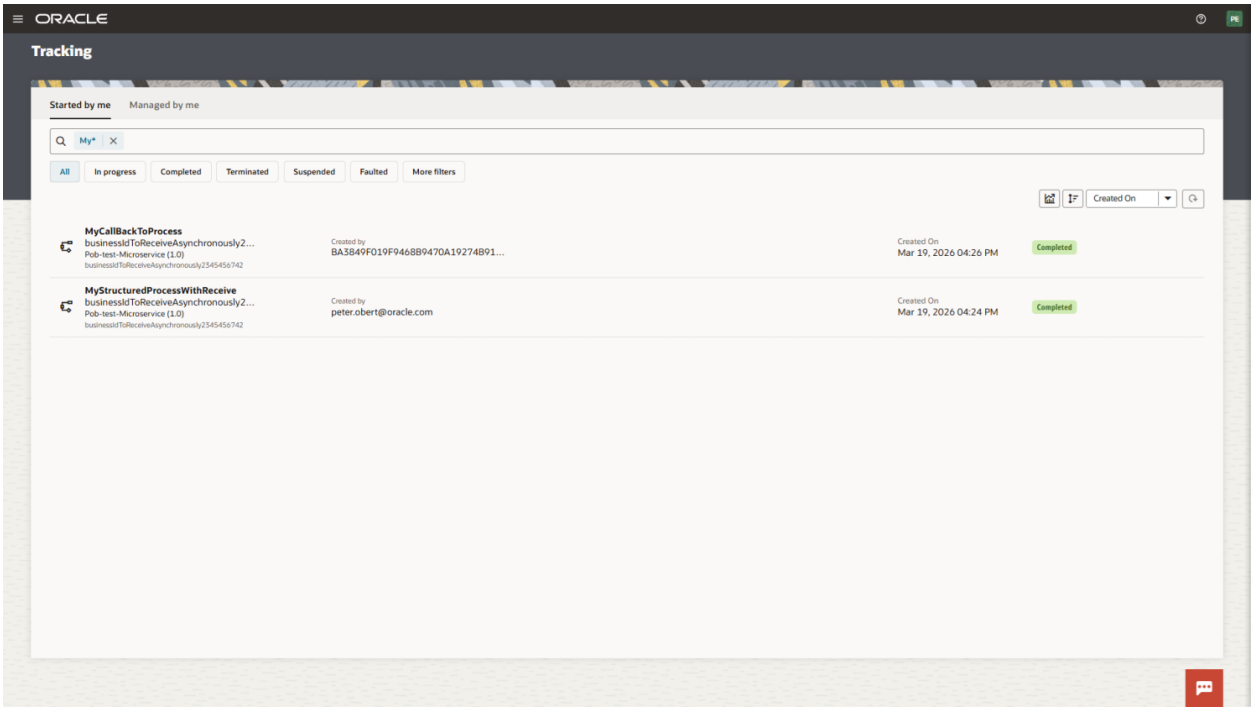


Example of the Synchronous and Asynchronous flow instances in the project

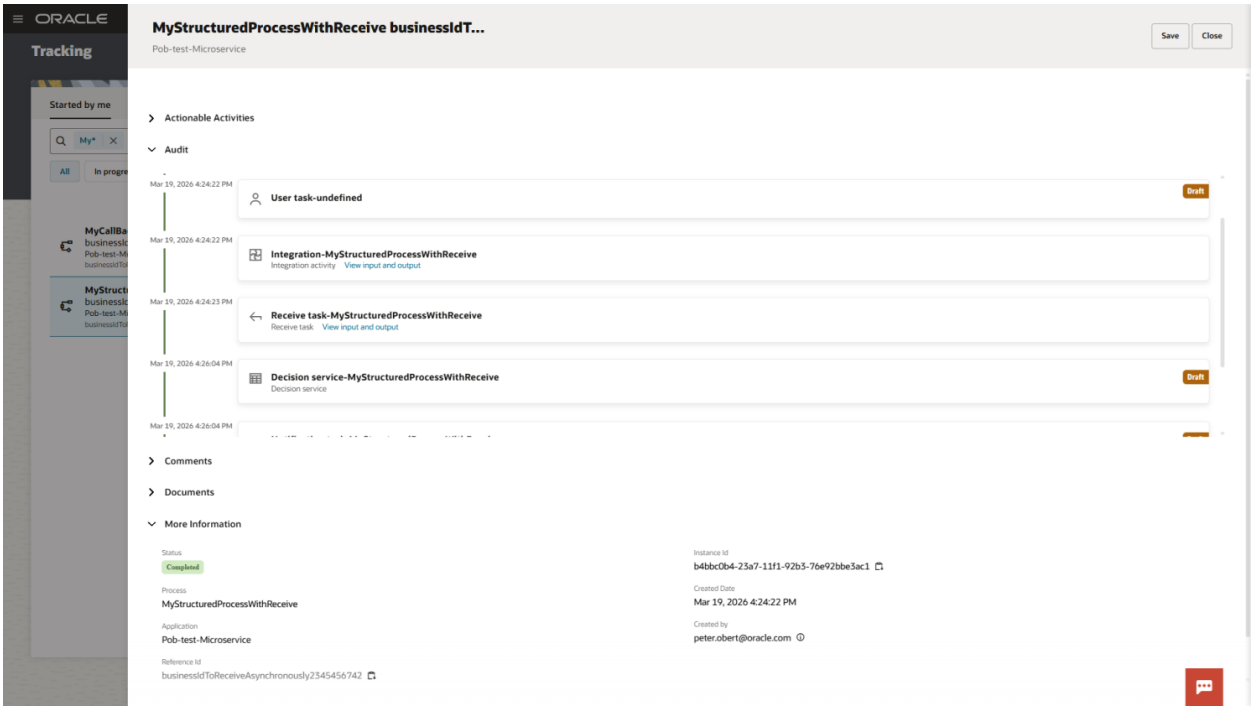
• Verify the Integration invokes the Callback Handler Flow on completion.



• Check Process Workspace: two process instances should show as Completed — the refactored business process and the Callback Handler.



• Confirm there are no timed-out instances in Process Workspace for the refactored process although business service call and task receive execution happens.



Notes

Why wrap business services in Integrations?
Isolating business service calls inside Integration orchestrations keeps your Process Automation flows technology-agnostic. Error handling, retries, and protocol-specific logic stay inside the integration layer, making the business process simpler and more maintainable.

Why not just raise the timeout limit?
Even if the OPA HTTP Connector timeout is increased beyond 90 seconds, synchronous design remains fragile for services that respond in minutes. An asynchronous pattern is inherently more resilient: the process instance persists and waits without holding a thread, and transient failures in the business service do not directly fault the process.

Correlation is critical
The Receive Task correlation properties are what route the callback payload back to the specific running process instance. Without correct correlation configuration, callbacks will either be lost or delivered to the wrong instance.

📦 **Templates**
Integration project and Process application archive templates are soon available in the Oracle Technology Engineering GitHub Repository: **github.com/oracle-devrel/technology-engineering**
Credits: **Niall Commiskey**, **Stan Tanev**